SOURCE-BACKED INTERVIEW FIELD GUIDE · 2026

# Explain AlgoMentor AI with confidence.

Architecture, request lifecycles, authentication, data, trade-offs, and the follow-up questions behind a 20–30 minute software engineering interview.

Next.js 15 + React 19

FastAPI + SQLAlchemy

PostgreSQL 16 + Redis 7

Gemini + coding platforms

Evidence comes from the local AlgoMentor source tree. Each technical claim is marked IMPLEMENTED, INFERRED, or RECOMMENDED. Code locations are included throughout. No live service behavior or production deployment was assumed.

01 / OPENING ANSWER

# Project in 60 seconds

IMPLEMENTED AlgoMentor AI is a competitive-programming coaching app. A learner links Codeforces or LeetCode, sees rating and topic analytics, receives a rule-generated study roadmap and problem recommendations, and can use a tutor and code reviewer backed by Gemini when configured.

**6**
protected feature route families

Profiles, analytics, roadmaps, tutor, code review, and predictions.

**11**
PostgreSQL tables

Users, tokens, profiles and derived learner artifacts.

**4**
Compose services

Frontend, backend, PostgreSQL, Redis.

IMPLEMENTED A visitor sees the public landing page, chooses register or login, then reaches a dashboard route. Client pages call FastAPI over JSON HTTP. The backend checks a bearer JWT and active user in PostgreSQL before running protected feature logic. Some endpoints read stored profile data; profile sync calls external coding services; tutor and code review may call Gemini.

`frontend/app/page.tsx:34–84 · frontend/app/login/page.tsx:13–30 · frontend/lib/api.ts:45–77 · backend/app/core/deps.py:18–45 · backend/app/api/v1/router.py:6–12`

## Interview introduction, three lengths

### 20 SECONDS

“AlgoMentor is a competitive-programming coach built with Next.js and FastAPI. It imports Codeforces and LeetCode profile data, turns it into topic analytics and study plans, and offers AI tutoring and code review. PostgreSQL stores learner data; JWTs protect the API.”

### 45 SECONDS

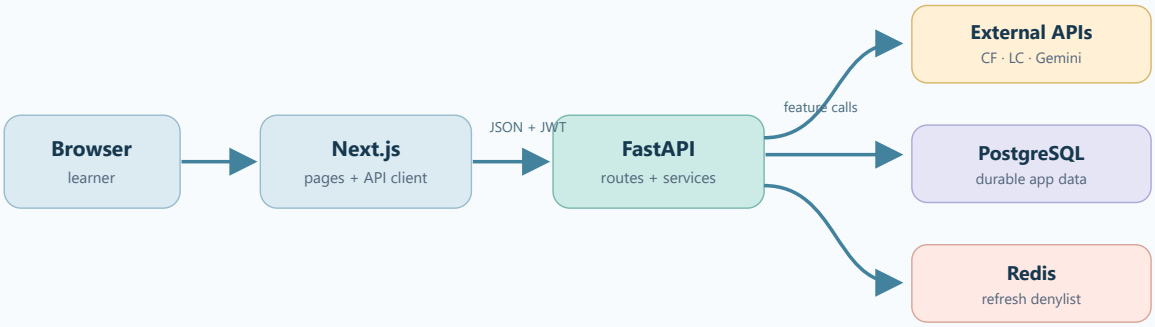
“I built AlgoMentor to help competitive programmers identify weak topics and plan practice. The Next.js client uses a typed API wrapper and TanStack Query. FastAPI routes delegate to services and SQLAlchemy repositories backed by PostgreSQL. Users sign in with bcrypt-hashed passwords and access/refresh JWTs. Profile sync pulls Codeforces and LeetCode data; analytics, recommendations, and rating predictions use the stored result. Gemini powers tutoring and code review when an API key is set.”

### 90 SECONDS

“The core user journey is register, sign in, link a coding profile, then inspect dashboard analytics and generate practice guidance. The frontend uses Next.js App Router with client components for interactive pages. A shared fetch client attaches an access JWT and refreshes it after a 401. FastAPI validates the token, loads the user, and invokes a service. Profile sync fetches Codeforces REST data plus LeetCode data through a third-party stats API and LeetCode GraphQL, normalizes submissions and topic tags, then replaces the profile's derived rows in PostgreSQL. Other services produce charts, rule-based roadmaps and recommendations, and request-time rating forecasts. The AI endpoints use Gemini if configured, with static fallback output otherwise. Redis currently tracks revoked refresh-token IDs; it does not cache dashboard data. I can also explain the limits: localStorage tokens, serial external calls during sync, synthetic LeetCode rating history, and development-focused Compose settings.”

02 / THE MAP

# Simple interview architecture




**IMPLEMENTED** Redis is used for refresh-token revocation, not a read-through data cache. Source: `backend/app/core/deps.py:52–60`.

## One-line design rationale

**INFERRED** The service and repository layers keep HTTP parsing, business logic, and database access separable, making feature code easier to test and change.

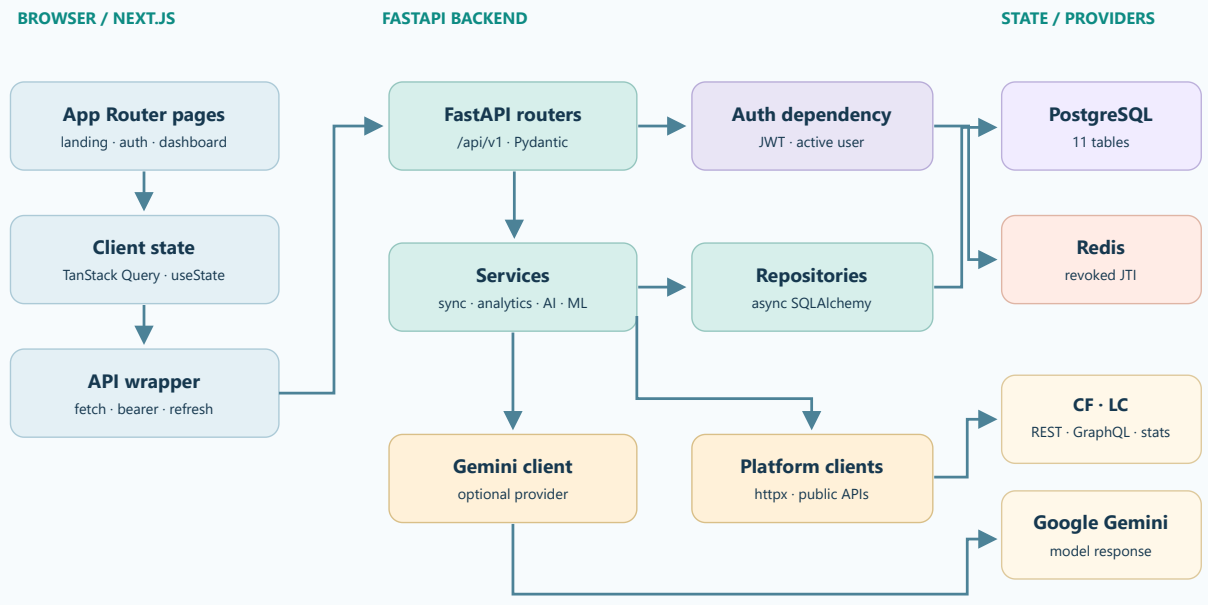
`backend/app/api/v1/profiles.py:13–27` · `backend/app/services/profile_service.py:13–24` · `backend/app/repositories/profile_repository.py:22–36`

## Accurate caveat

**IMPLEMENTED** "AI-powered" is conditional. If `GEMINI_API_KEY` is absent, the Gemini client returns built-in fallback responses.

`backend/app/external/gemini_client.py:9–23`

## Detailed engineering diagram




Read left to right for a normal request, then down for external integrations. Auth is a dependency within the FastAPI process, not a separate deployed service.

03 / CODE MAP

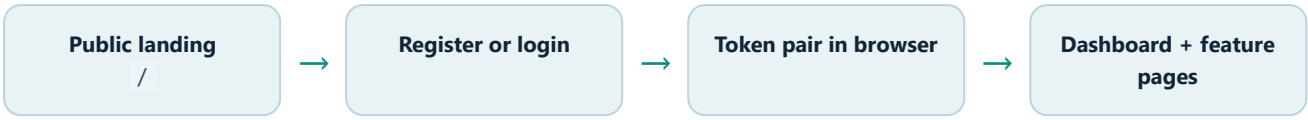
# Folders worth remembering

| Directory | Purpose and connection | Interview anchor |
| --- | --- | --- |
| `frontend/app` | **IMPLEMENTED** App Router pages: public landing, register/login, and the `(dashboard)` route group. The dashboard layout checks local token presence. | `app/(dashboard)/layout.tsx:8-15` |
| `frontend/lib` | **IMPLEMENTED** Shared HTTP wrapper, token storage, automatic refresh and endpoint functions. | `lib/api.ts:17-141` |
| `frontend/components` | **IMPLEMENTED** Query provider, navigation, charts, and small UI primitives. | `components/providers.tsx:6-16` |
| `backend/app/api/v1` | **IMPLEMENTED** Route declaration, Pydantic request/response binding, auth dependency, service dispatch. | `api/v1/router.py:6-12` |
| `backend/app/core` | **IMPLEMENTED** Environment settings, JWT/password functions, bearer dependency, Redis client. | `core/deps.py:18-60` |
| `backend/app/services` | **IMPLEMENTED** Business rules for auth, profile ingestion, analytics, roadmaps, tutor, review, recommendations, predictions. | `services/profile_service.py:13-128` |
| `backend/app/repositories` | **IMPLEMENTED** SQLAlchemy query and mutation methods, called by services. | `repositories/profile_repository.py:22-183` |
| `backend/app/models + schemas` | **IMPLEMENTED** ORM tables versus API validation and serialization contracts. | `models/user.py:13-197` |
| `backend/app/external` | **IMPLEMENTED** Codeforces, LeetCode and Gemini adapters. | `external/platform_clients.py:8-175` |
| `backend/alembic + seed` | **IMPLEMENTED** Initial schema migration and demo/catalog seed invoked at Compose startup. | `alembic/versions/001_initial_schema.py:19-190` |

> **Five files to memorize:** `frontend/lib/api.ts` → `backend/app/main.py` → `backend/app/core/deps.py` → `backend/app/services/profile_service.py` → `backend/app/models/user.py`. Together they explain client transport, API entry, authorization, core ingestion, and persistence.

04 / RUNTIME BEHAVIOR

# Frontend, state, and the user journey




IMPLEMENTED **Rendering and navigation.** The landing page is an App Router page with links to login and registration. Interactive pages are client components. The root layout mounts `Providers`; the dashboard route group renders a sidebar and redirects to login after mount if no access token is present. This client check is a UX guard, not the authorization boundary.

`frontend/app/page.tsx:1–110` · `frontend/app/layout.tsx:13–21` · `frontend/app/(dashboard)/layout.tsx:1–25`

IMPLEMENTED **State.** TanStack Query holds server-fetched data. Its default stale time is 60 seconds with one retry. Pages use `useQuery` for reads and `useMutation` for writes; profile sync invalidates the profile and dashboard queries. React `useState` holds form fields, tutor messages, selected session, and similar local UI state. Access and refresh tokens live in `localStorage`.

`frontend/components/providers.tsx:6–16` · `frontend/app/(dashboard)/profile/page.tsx:14–36` · `frontend/app/(dashboard)/tutor/page.tsx:21–60` · `frontend/lib/api.ts:17–34`

IMPLEMENTED **Transport.** `apiFetch` adds JSON headers and bearer auth, calls the browser-reachable `NEXT_PUBLIC_API_URL`, refreshes tokens after a 401 if an access token was sent, retries once, and throws the API's `detail` on remaining errors. Logout clears local tokens in a `finally` block even if the server call fails.

`frontend/lib/api.ts:38–97` · `frontend/components/layout/sidebar.tsx:34–42`

### What happens after login?

IMPLEMENTED The login page stores the returned token pair and navigates to `/dashboard`. The dashboard calls `GET /analytics/dashboard`. If no profile exists, the API returns 404 and the UI offers a link to profile setup.

`frontend/app/login/page.tsx:20–29` · `frontend/app/(dashboard)/dashboard/page.tsx:11–34` · `backend/app/services/analytics_service.py:24–33`

### What is not server state?

IMPLEMENTED The tutor page keeps displayed chat messages in local component state, though session history is persisted in PostgreSQL. The API client does not use Next.js server actions or Next.js API routes.

`frontend/app/(dashboard)/tutor/page.tsx:21–59` · `backend/app/services/tutor_service.py:20–70` · `frontend/lib/api.ts:45–77`

---

05 / REQUEST TRACES

# Two complete request lifecycles

## A. Authenticated dashboard read

1. IMPLEMENTED `DashboardPage` starts the `["dashboard"]` query, which calls `analyticsApi.dashboard()`.
2. IMPLEMENTED `apiFetch` reads the access token from `localStorage`, attaches `Authorization: Bearer …`, and fetches `/api/v1/analytics/dashboard`.
3. IMPLEMENTED FastAPI's CORS middleware handles allowed browser origins. The route asks `get_current_user` to decode an access JWT and fetch an active user from PostgreSQL.
4. IMPLEMENTED `AnalyticsService` loads the user's profile, submissions, rating history and topic scores, computes stats, chart points and heatmap cells, and returns a `DashboardResponse`.
5. IMPLEMENTED With no profile, the service raises 404. With an expired access token, the API returns 401 and `apiFetch` attempts token refresh before retrying.

`frontend/app/(dashboard)/dashboard/page.tsx:11–47` · `frontend/lib/api.ts:45–77` · `backend/app/main.py:23–30` · `backend/app/api/v1/analytics.py:13–18` · `backend/app/core/deps.py:18–45` · `backend/app/services/analytics_service.py:23–76`

> IMPLEMENTED This is a PostgreSQL-backed read. Redis is not consulted on dashboard requests. The previous browser → Redis → PostgreSQL cache-miss diagram describes a possible enhancement, not present behavior.

## B. Profile sync write

1. IMPLEMENTED The profile form sends `POST /profiles/sync` with one or both handles; the route requires an authenticated user.

2. **IMPLEMENTED** `ProfileService.link_and_sync` validates that at least one handle is provided, loads or creates the one-to-one profile, and calls configured platform clients.
3. **IMPLEMENTED** Codeforces calls `user.info`, `user.rating`, and `user.status` in sequence, keeping up to 500 submissions. LeetCode calls the third-party `leetcode-stats-api.herokuapp.com` and then LeetCode GraphQL.
4. **IMPLEMENTED** The service normalizes tags, builds ratings and submission objects, computes topic scores, deletes prior child rows, adds the new rows, and returns profile + scores. The DB dependency commits after successful handling and rolls back on exception.
5. **IMPLEMENTED** The frontend invalidates profile and dashboard queries so the next read sees the updated data.

`frontend/app/(dashboard)/profile/page.tsx:18-37` · `backend/app/api/v1/profiles.py:21-27` · `backend/app/services/profile_service.py:25-128` · `backend/app/external/platform_clients.py:8-175` · `backend/app/db/session.py:9-16`

> **IMPLEMENTED** The LeetCode adapter constructs a synthetic rating series in `build_synthetic_rating_history`; this is not real contest history. Also, when both platforms are linked, `total_solved` is set from LeetCode statistics rather than a combined unique count. Explain this honestly if asked about data quality.
>
> `backend/app/external/platform_clients.py:157-175` · `backend/app/services/profile_service.py:47-70`

---

06 / IDENTITY BOUNDARY

# Authentication and authorization

### Registration + login

**IMPLEMENTED** Registration validates email/username/password with Pydantic, checks uniqueness, hashes the password using passlib/bcrypt, and inserts the user. Login verifies the hash and active flag, signs JWTs with HS256 by default, and stores the refresh `jti` in PostgreSQL.

`backend/app/schemas/auth.py:7-18` · `backend/app/services/auth_service.py:26-54` · `backend/app/core/security.py:10-35`

### Protected endpoints

**IMPLEMENTED** `get_current_user` requires a Bearer token, decodes it, checks `type=access`, and loads an active user. Each feature route declares this dependency. No roles or permissions beyond "active authenticated user" are modeled.

`backend/app/core/deps.py:18-45` · `backend/app/api/v1/*.py`

### Token lifecycle in one picture




**IMPLEMENTED** Refresh validates JWT type, Redis denylist status, stored token expiry/revocation, and active user; it revokes and blacklists the old token before creating a new pair. Logout revokes the supplied refresh token and blacklists its `jti`. The Redis TTL is hard-coded to seven days, and the stored refresh expiry is also hard-coded to seven days even though JWT expiry reads `REFRESH_TOKEN_EXPIRE_DAYS`; changing that setting can make them disagree.

`backend/app/services/auth_service.py:55-100` · `backend/app/core/security.py:28-35` · `backend/app/core/deps.py:52-60` · `backend/app/core/config.py:10-14`

**IMPLEMENTED** The frontend stores both tokens in `localStorage`. A dashboard layout redirect checks only token presence after render. Server authorization relies on `get_current_user`; no custom Next.js middleware or role-based access control is present. Access tokens are not checked against the Redis denylist, so logout does not immediately invalidate an already issued access token.

`frontend/lib/api.ts:17-34` · `frontend/app/(dashboard)/layout.tsx:8-15` · `backend/app/core/deps.py:18-45`

> **Interview distinction:** Authentication asks "who is this user?"; authorization asks "may they access this resource?" Here, the general rule is active authenticated user, while tutor session lookup also scopes by `ChatSession.user_id == user.id` and feature queries filter by the current user.
>
> `backend/app/repositories/profile_repository.py:111-126` · `backend/app/services/tutor_service.py:28-39`

---

07 / PERSISTENCE

# Database architecture

**IMPLEMENTED** PostgreSQL is the durable system of record. SQLAlchemy async sessions are created per request through `get_db`; Alembic has one initial migration. UUID primary keys identify most rows, foreign keys connect users to profiles, tokens, roadmaps, chats and reviews, and a unique `competitive_profiles.user_id` enforces one profile per user.

`backend/app/db/session.py:5-16` · `backend/app/models/user.py:13-197` · `backend/alembic/versions/001_initial_schema.py:19-190`

| Area | Tables and relationships | Why it exists |
|---|---|---|
| Identity | `users` 1:N `refresh_tokens` | Accounts and tracked refresh JTIs |
| Profile | `users` 1:1 `competitive_profiles` | Linked handles and aggregate metrics |
| History | `competitive_profiles` 1:N `rating_history`, `submissions`, `topic_scores` | Analytics and weakness calculations |
| Learning | `users` 1:N `roadmaps`, `chat_sessions`, `code_reviews`; `chat_sessions` 1:N `chat_messages` | Saved plans, conversations, analyses |
| Catalog | `problem_catalog` | Seeded recommendation candidates |

**IMPLEMENTED** JSONB stores flexible raw profile metadata, topic lists, roadmap plans, chat metadata, review analysis, and catalog tags. Email and username have unique indexes; refresh `jti` is unique and indexed; profile child foreign keys are indexed in the ORM model. Profile sync deletes and reinserts child rows rather than upserting individual submissions.

`backend/app/models/user.py:16-197` · `backend/app/repositories/profile_repository.py:38-57` · `backend/app/services/profile_service.py:77-118`

**RECOMMENDED** For larger histories, add uniqueness constraints for platform/problem/contest identities, batch upserts, and indexes that match frequent user/time filters. Consider a durable sync job with an explicit status table instead of keeping external HTTP calls inside a request-scoped database session.

---

08 / HTTP SURFACE

# Important APIs: what to say and what to watch

All paths below are relative to `/api/v1` unless noted. Every row describes implemented code. Feature routes use `get_current_user`; auth endpoints are public.

| Method + path | Operation / response | Important behavior | Source |
|---|---|---|---|
| `POST /auth/register` | Create user; 201 user | bcrypt hash; duplicate email/username → 400 | `auth.py:13–16` |
| `POST /auth/login` | Issue access + refresh pair | bad password → 401; inactive → 403 | `auth.py:18–20` |
| `POST /auth/refresh` | Rotate refresh pair | Redis denylist + DB row + active user checks | `auth.py:23–25` |
| `POST /auth/logout` | Revoke refresh; 204 | access token remains valid until expiry | `auth.py:28–30` |
| `GET /profiles/me` | Current linked profile or null | read by authenticated user ID | `profiles.py:13–19` |
| `POST /profiles/sync` | Import profile + topic scores | external HTTP + replace child rows | `profiles.py:21–27` |
| `GET /analytics/dashboard` | Stats, charts, heatmap | 404 until profile exists | `analytics.py:13–18` |
| `POST /roadmaps/generate`<br>`GET /roadmaps` | Create/list study plans | rule-generated from weak topics; duration 4–12 weeks | `roadmaps.py:15–29` |
| `POST /tutor/sessions`<br>`GET /tutor/sessions`<br>`GET /tutor/sessions/{id}`<br>`POST /tutor/sessions/{id}/messages` | Persist chat sessions and send message | session lookup scoped to current user; Gemini or fallback | `tutor.py:15–48` |
| `POST /code-review`<br>`GET /code-review/history` | Analyze code and list history | stores source code and analysis JSONB | `code_review.py:13–27` |
| `GET /predictions/ratings` | Next contest, 3/6-month estimates | per-request Ridge fit with enough history; heuristic otherwise | `predictions.py:14–20` |
| `GET /predictions/interview-readiness` | Four company-labelled scores | hard-coded weighted formula, not recruiter data | `predictions.py:22–28` |
| `GET /predictions/recommendations` | Easy/medium/hard problem lists | scores catalog by topic overlap and rating fit | `predictions.py:30–35` |
| `GET /health` | Static healthy response | does not probe PostgreSQL or Redis | `main.py:33–37` |

**IMPLEMENTED** “Rating prediction” combines a request-time Ridge regression with fallback heuristics. “Interview readiness” is a deterministic weighted score. “Success probability” in recommendations is a hand-built score. Do not describe any of these as validated predictive models.

`backend/app/services/prediction_service.py:14–114` · `backend/app/services/recommendation_service.py:25–81`

09 / BOUNDARIES AND FAILURES

# External services, middleware, errors

| Dependency | Actual call path | Failure or fallback |
| --- | --- | --- |
| Codeforces | **IMPLEMENTED** Three public REST calls: `user.info`, `user.rating`, `user.status`, each via `httpx` with 30s timeout. | HTTP errors propagate; a non-OK `user.info` response raises `ValueError`. No retry or circuit breaker. |
| LeetCode | **IMPLEMENTED** GET to `leetcode-stats-api.herokuapp.com/{username}`, then POST to `leetcode.com/graphql`. | Stats 404 raises `ValueError`; other stats HTTP errors propagate. A non-200 GraphQL response is treated as empty data. |
| Google Gemini | **IMPLEMENTED** `gemini-1.5-flash` via `google-generativeai`; tutor and code review call the adapter. | If no key, returns static fallback JSON. Runtime provider exceptions are not caught by the adapter. |
| Redis | **IMPLEMENTED** `blacklist:refresh:{jti}` with `setex` / `exists` during refresh and logout. | No application-data cache. Redis errors are not handled in these functions. |

`backend/app/external/platform_clients.py:8–31,77–115` · `backend/app/external/gemini_client.py:9–23` · `backend/app/core/deps.py:52–60`

## Middleware and error taxonomy

**IMPLEMENTED** `CORSMiddleware` is the only middleware explicitly registered by the app; it allows configured origins, all methods/headers and credentials. Authentication is a FastAPI dependency, not middleware. No custom exception handler or Next.js middleware file appears in the source tree.

`backend/app/main.py:18–31` · `backend/app/core/config.py:17–23` · `backend/app/core/deps.py:18–45`

### Handled business errors

**IMPLEMENTED** Auth raises 400/401/403 for duplicates, invalid credentials/tokens and disabled accounts; profile, roadmap and recommendation services raise 400 or 404 for missing inputs/profile; Pydantic/FastAPI handles schema validation with 422.

`backend/app/services/auth_service.py:26–88` · `backend/app/services/profile_service.py:25–32` · `backend/app/services/roadmap_service.py:27–30` · `backend/app/schemas/*.py`

### Unnormalized failures

**INFERRED** Raw `httpx`, provider, Redis or parsing errors are likely surfaced as server errors because no route-level mapping or global handler is defined. Client `apiFetch` falls back to “Request failed” if an error body is not JSON.

`backend/app/external/platform_clients.py:11–30,81–108` · `backend/app/main.py:1–37` · `frontend/lib/api.ts:70–74`

**IMPLEMENTED** The `/health` endpoint returns a fixed payload and does not prove the database, Redis, or Gemini are available. Compose has container health checks only for PostgreSQL and Redis.

`backend/app/main.py:33–37` · `docker-compose.yml:11–30`

10 / SECURITY REVIEW

# Security: what is true today

| Topic | Current implementation | Interview-ready assessment |
|---|---|---|
| Password storage | **IMPLEMENTED** passlib/bcrypt hash, never plain password in user table. | Good baseline; add login rate limiting and password policy if exposed. |
| JWT validation | **IMPLEMENTED** Signed HS256 token, expiry and type checked; active user fetched on protected requests. | Short access lifetime reduces—but does not remove—post-logout exposure. |
| Refresh rotation | **IMPLEMENTED** Old row revoked, JTI denylisted in Redis, new row written. | Concurrent refreshes can race because the check and revoke are not an atomic compare-and-swap. |
| Browser token storage | **IMPLEMENTED** Access and refresh tokens in `localStorage`. | JavaScript-readable storage increases impact of an XSS bug. Consider secure HttpOnly cookies with CSRF defenses. |
| Secrets and demo data | **IMPLEMENTED** Example JWT secret/password, Compose defaults and a known seeded demo credential. | Replace values and remove public demo seeding before a real deployment. |
| Resource ownership | **IMPLEMENTED** Tutor session queries include the current user ID; profile/recommendation data is fetched by user ID. | Useful object-level authorization pattern. Keep testing ownership on every new resource endpoint. |

`backend/app/core/security.py:10–41` · `backend/app/services/auth_service.py:55–100` · `frontend/lib/api.ts:17–34` · `.env.example:1–23` · `backend/app/seed/run.py:36–53` · `backend/app/repositories/profile_repository.py:111–126`

## Three specific findings to volunteer if asked

1. **IMPLEMENTED** Refresh JWT expiry uses `REFRESH_TOKEN_EXPIRE_DAYS`, but database expiry and Redis TTL are fixed at seven days. If configuration changes, the three authorities can disagree.
   `backend/app/core/security.py:28–35` · `backend/app/services/auth_service.py:71–100`
2. **INFERRED** Two near-simultaneous refresh calls can both read the same not-yet-revoked row before either commit, allowing duplicate replacement tokens. The code has no row lock or atomic conditional update.
   `backend/app/services/auth_service.py:67–81` · `backend/app/repositories/profile_repository.py:64–76` · `backend/app/db/session.py:9–16`
3. **IMPLEMENTED** There is no server-side access-token revocation path. The bearer dependency does not query Redis, and logout only revokes refresh tokens.
   `backend/app/core/deps.py:18–45` · `backend/app/services/auth_service.py:83–95`

> **RECOMMENDED** Production hardening order: rotate secrets and disable demo seeding; move refresh token to a secure cookie with a clear CSRF design; add login throttling; make refresh rotation atomic; centralize provider error mapping, timeouts, and observability. The current repository does not implement these controls.

---

11 / DEPLOYMENT AND GROWTH

# Deployment architecture, scaling, bottlenecks

**IMPLEMENTED** Compose defines four containers and named volumes for PostgreSQL and Redis. Backend startup runs Alembic migration, the seed script, then `uvicorn --reload`. Frontend runs `npm run dev`. Both source trees are bind-mounted, and ports 3000, 8000, 5432 and 6379 are published. This is a local-development deployment, not evidence of a production release.

`docker-compose.yml:1–78` · `frontend/Dockerfile:1–11` · `backend/Dockerfile:1–14`

**IMPLEMENTED** `next.config.ts` sets `output: "standalone"`, but the Docker command still runs the development server. The backend exposes an API docs page through FastAPI by default; no load balancer, queue, Kubernetes manifest, CI/CD pipeline, or production infrastructure is present in the inspected repository.

`frontend/next.config.ts:1–7` · `docker-compose.yml:34–71` · `backend/app/main.py:18–37`

> **IMPLEMENTED** **Configuration to know:** `DATABASE_URL` and `REDIS_URL` select storage; `JWT_SECRET_KEY` signs tokens; `GEMINI_API_KEY` enables live AI; `BACKEND_CORS_ORIGINS` controls browser origins; `NEXT_PUBLIC_API_URL` is the browser-visible API base, not a secret. Compose overrides the backend database and Redis URLs with container hostnames. Codeforces and LeetCode calls do not use API keys in this code.
> `.env.example:1–23` · `backend/app/core/config.py:5–23` · `docker-compose.yml:37–42` · `frontend/lib/api.ts:1`

| Bottleneck / limit | Code evidence | Practical next step |
| --- | --- | --- |
| IMPLEMENTED Profile sync waits on serial external requests inside an API call. | `platform_clients.py:11-30,81-108` ; `profile_service.py:38-65` | RECOMMENDED Background sync jobs, concurrency where safe, retries with backoff, status endpoint. |
| IMPLEMENTED Large syncs delete/recreate all child rows; Codeforces handles up to 500 recent submissions. | `platform_clients.py:26-31` ; `profile_service.py:77-118` | RECOMMENDED Incremental imports and dedupe keys. |
| IMPLEMENTED Every protected request loads the user from PostgreSQL. | `core/deps.py:35-45` | RECOMMENDED Profile DB traffic before optimizing; optional short-lived identity cache with invalidation. |
| IMPLEMENTED Recommendations load all catalog rows and sort in Python. | `repositories/profile_repository.py:171-183` ; `recommendation_service.py:31-81` | RECOMMENDED Index/filter/paginate when catalog grows. |
| IMPLEMENTED Ridge regression is fitted per prediction request when history is long enough. | `prediction_service.py:14-59` | RECOMMENDED Train/evaluate offline or cache model outputs; show uncertainty honestly. |
| IMPLEMENTED Gemini model calls are awaited during tutor/review requests. | `external/gemini_client.py:17-23` ; `tutor_service.py:44-45` | RECOMMENDED Provider timeout, budget limits, queue or streaming, structured output validation. |

> RECOMMENDED **Scaling story to tell:** move to production images and managed PostgreSQL/Redis; isolate long profile-sync and AI tasks in workers; add status/polling; use connection pools, targeted indexes, backpressure and metrics; consider response caching only after measuring dashboard read load and defining invalidation on sync. None of these are deployed here.

12 / INTERVIEW DRILL

# Questions, strong answers, deeper follow-ups

**1. Why Next.js plus FastAPI?**

INFERRED "Next.js gives me route-based UI composition and client-side interaction; FastAPI hosts the Python services for data ingestion, analytics and AI. The boundary is JSON REST, so each side can evolve independently."

Follow-up: What costs come with two runtimes? → Two deployables, CORS, shared API contracts and cross-origin token handling. `frontend/lib/api.ts:1-77` ; `backend/app/main.py:18-31` .

**2. Trace login end to end.**

IMPLEMENTED "The form calls `/auth/login` . The service gets the user by email, verifies bcrypt hash and active status, signs access/refresh JWTs, persists refresh JTI, then returns the pair. The browser stores both and sends the access token on later requests."

Follow-up: What happens after access expiry? → 401 triggers one refresh and retry. `auth_service.py:42-54,96-100` ; `frontend/lib/api.ts:38-77` .

**3. Is the dashboard cached in Redis?**

IMPLEMENTED "No. Redis only stores refresh-token denylist entries. Dashboard analytics reads PostgreSQL via repositories and computes the response."

Follow-up: When would you cache it? → Measure repeated reads, choose a short TTL, and invalidate on profile sync. `core/deps.py:52-60` ; `analytics_service.py:23-76` .

**4. How do you prevent users reading other users' tutor sessions?**

IMPLEMENTED "The bearer dependency establishes the active user, and `ChatRepository.get_session` filters by both session ID and that user's ID."

Follow-up: Does every endpoint have ownership checks? → Review each new resource query; general authentication alone is not enough. `profile_repository.py:111-126` .

**5. How does profile data become a roadmap?**

IMPLEMENTED "Sync normalizes accepted submissions and tags into topic scores. The roadmap service selects the weakest topics, cycles them across requested weeks, combines a rating gap and curated resources, then saves the plan JSON."

Follow-up: Is the roadmap AI-generated? → No; it is deterministic service logic. `topic_analysis_service.py:4-44` ; `roadmap_service.py:27-86` .

**6. What exactly does the ML model predict?**

IMPLEMENTED "With at least three rating-history points, it fits a Ridge model in that request to predict rating delta; otherwise it uses a heuristic. The 3- and 6-month values extrapolate that delta. These are illustrative estimates, not validated forecasts."

Follow-up: How would you validate it? → Historical train/test splits by user and time, MAE/calibration against baselines. `prediction_service.py:14-59,104-114` .

**7. How is a refresh token revoked?**

IMPLEMENTED "The service marks its PostgreSQL row revoked and writes the JTI to a Redis denylist. Refresh checks both before issuing a new pair."

Follow-up: Is rotation race-safe? → Not fully; an atomic conditional update or row lock would help. `auth_service.py:55-100` .

**8. What happens when Gemini is unavailable?**

IMPLEMENTED "With no configured key, the adapter returns static fallback JSON. If a configured provider call fails at runtime, there is no explicit exception mapping in that adapter."

Follow-up: How to improve? → Set timeout/retry budget, distinguish unavailable from fallback, validate structured output. `gemini_client.py:9-23,25-68` .

**9. What is the most expensive request?**

INFERRED "Profile sync is a likely hotspot because it waits for multiple external HTTP calls, parses submissions, deletes old rows and reinserts data before responding. I would measure latency and move it to a job if it dominates."

Follow-up: What if a provider fails mid-sync? → The DB session rolls back, but external latency and errors still affect the request. `profile_service.py:25-128` ; `db/session.py:9-16` .

**10. What is the most important security change?**

RECOMMENDED "First rotate the example credentials and disable public demo seeding. Then address JavaScript-readable refresh tokens, login throttling and atomic refresh rotation."

Follow-up: Why not simply put all tokens in cookies? → Cookie transport needs SameSite, Secure, HttpOnly and a CSRF design. `.env.example:1-23` ; `frontend/lib/api.ts:17-34` .

**11. What if Redis goes down?**

INFERRED "Normal access-token-protected feature requests still validate JWT and load the user from PostgreSQL. Refresh/logout would likely fail because they call Redis without a fallback."

Follow-up: Should refresh fail open? → I would fail closed for revocation checks and return a controlled 503. `core/deps.py:52-60` ; `core/deps.py:18-45` .

**12. How does deployment work today?**

IMPLEMENTED "Docker Compose starts PostgreSQL and Redis, then the backend runs migrations and seed before Uvicorn, and Next.js serves the frontend. The commands are development settings, including reload and the dev server."

Follow-up: Production path? → Build immutable images, remove bind mounts/reload/seed, put services behind TLS and managed stateful infrastructure. `docker-compose.yml:1-78` .

**13. How do you handle errors across layers?**

IMPLEMENTED "Services raise FastAPI HTTP errors for known business cases; Pydantic validates request bodies; the browser wrapper parses an error `detail` or a generic fallback. Provider and infrastructure exceptions are not centrally normalized yet."

Follow-up: What would you add? → Typed error contract, exception handlers, correlation IDs and logs. `auth_service.py:26-88` ; `frontend/lib/api.ts:70-74` .

**14. What data is synthetic or heuristic?**

IMPLEMENTED "LeetCode rating-history points are generated from the current rating; recommendation 'success probability' is a formula; interview readiness is a weighted score. I would label these estimates clearly in the product and avoid calling them calibrated probabilities."

Follow-up: What real data would improve them? → Historical attempts, contest outcomes and validated labels. `platform_clients.py:157-175` ; `recommendation_service.py:36-81` ; `prediction_service.py:61-101` .

---

13 / DESIGN ANSWER

# If asked, "What would you improve next?"

### 1. Correctness and trust

RECOMMENDED Mark synthetic LeetCode history as illustrative or remove it from real charts; fix combined solved counts; add contract tests for platform parsers and response schemas; avoid presenting heuristic scores as calibrated probabilities.

### 2. Reliability

RECOMMENDED Move profile sync to a durable job, show progress and failure states, add provider timeouts/retries, preserve the last good profile during an upstream outage, and map failures to useful API errors.

### 3. Security

**RECOMMENDED** Use deployment secrets, disable demo account seeding, add login throttling, make token rotation atomic, and plan a secure cookie-based refresh flow with CSRF controls.

### 4. Production operations

**RECOMMENDED** Build production images, remove `--reload` and source bind mounts, add test/CI gates, structured logs and metrics, dependency health probes, database backups, and migration rollout discipline.

## 30-second improvement answer

> "I would first fix trust and reliability. Some LeetCode history is synthetic, and profile sync blocks on several external calls, so I would label the data accurately, add parser tests, and move sync into a durable job with a status endpoint. Then I would harden auth by rotating example secrets, removing public demo seeding, and making refresh rotation atomic. Once the behavior is measured, I would scale specific hotspots rather than add caching everywhere."

Scope note: This guide is a static source audit of the local workspace. The directory has no Git metadata, so it cannot cite a pinned commit. There were no first-party test files or CI workflows in the inspected tree. The guide does not claim a successful runtime integration test or production deployment.